

An Overview of a Compiler for Scalable Parallel Machines

Saman P. Amarasinghe, Jennifer M. Anderson,
Monica S. Lam and Amy W. Lim

Computer Systems Laboratory
Stanford University, CA 94305

Abstract. This paper presents an overview of a parallelizing compiler to automatically generate efficient code for large-scale parallel architectures from sequential input programs. This research focuses on loop-level parallelism in dense matrix computations. We illustrate the basic techniques the compiler uses by describing the entire compilation process for a simple example.

Our compiler is organized into three major phases: analyzing array references, allocating the computation and data to the processors to optimize parallelism and locality, and generating code.

An optimizing compiler for scalable parallel machines requires more sophisticated program analysis than the traditional data dependence analysis. Our compiler uses a precise data-flow analysis technique to identify the producer of the value read by each instance of a read access. In order to allocate the computation and data to the processors, the compiler first transforms the program to expose loop-level parallelism in the computation. It then finds a decomposition of the computation and data such that parallelism is exploited and the communication overhead is minimized. The compiler will trade off extra degrees of parallelism to reduce or eliminate communication. Finally, the compiler generates code to manage the multiple address spaces and to communicate data across processors.

1 Introduction

A number of compiler systems, such as SUPERB[21], AL[17], ID Noveau[15], Kali[11], Vienna FORTRAN[3], FORTRAN D[8, 16] and HPF[7], have been developed to make effective use of scalable parallel machines. These systems simplify the compilation problem by soliciting the programmer's help in determining

This research was supported in part by DARPA contracts N00039-91-C-0138 and DABT63-91-K-0003, an NSF Young Investigator Award and fellowships from Digital Equipment Corporation's Western Research Laboratory and Intel Corporation.

In Proceedings of the Sixth Workshop on Languages and Compilers for Parallel Computing, Portland, OR, Aug. 12-14, 1993. To be published by Springer-Verlag as Lecture Notes in Computer Science.

the data decompositions. The goal of our research is to fully automate the parallelization process. We are currently developing a compiler to automatically translate sequential scientific programs into efficient parallel code.

Our compilation techniques are applicable to all machines with non-uniform memory access times. Our target machines include both distributed address space machines such as the Intel iPSC/860[9], and shared address space machines such as the Stanford DASH multiprocessor[12]. Our research focuses on dense matrix computations written in sequential FORTRAN-77, though many of the techniques presented are also applicable to optimizations within and especially across FORTRAN-90 statements.

Our techniques are applicable to programs with arbitrary nestings of parallel and sequential loops. The loop bounds and array subscripts must be affine functions of the loop indices and symbolic constants. Our techniques can also handle conditional statements, although less precise analysis techniques must be used in these cases. We characterize parallel machines by their message passing overhead, message latency and communication bandwidth; machine topology is not considered significant. Currently, our compiler operates on each procedure individually, and cannot handle function calls.

We have decomposed the complex problem of compiling for large-scale parallel machines into several well-defined subproblems: extracting precise array data-flow information from the program, mapping the data and computation onto the processors and generating efficient code. Algorithms to solve these subproblems have been developed and implemented in our SUIF (Stanford University Intermediate Format) compiler. This paper presents an overview of our parallelizing compiler, concentrating on how the various components fit together. We explain the steps taken in the compiler by showing how they operate on a simple example. We assume that readers are familiar with the literature on parallelizing compilers. Details and related work on individual algorithms can be found in our other papers[1, 2, 14, 18, 19].

An optimizing compiler for scalable parallel machines needs more sophisticated program analysis than the traditional data dependence analysis. Exact array data-flow analysis determines if two dynamic instances of an array access refer to the same *value*, and not just to the same *location*. This information is useful for array privatization, an important optimization for increasing parallelism. It is also important for generating efficient code for distributed address space machines.

Reducing the frequency of synchronization is critical to minimizing communication overhead. Our first step in the parallelization process is to expose the maximum degree of coarse-grain parallelism in the code. Loop transforms such as unimodular transforms (combinations of loop interchange, skewing and reversal), loop tiling and array privatization are used to achieve this goal.

The decomposition phase maps the data and computation onto the processors of the machine. We observe that the choice of parallelism can affect the communication cost. In addition to aligning the data and computation to minimize communication, our algorithm will also choose to trade off excess parallelism if

necessary.

Our parallelization and decomposition algorithms treat the code within the innermost loop as an atomic unit. It is possible to improve the parallelization by expanding our scope to include optimizations that can alter the composition of these atomic units. Such transformations include loop reindexing, fission and fusion.

The final phase of the compiler generates an SPMD (single program multiple data) program from the decompositions produced in the previous phase. For distributed address space machines, this phase needs to generate the necessary receive and send instructions, allocate memory locally on each processor and translate global data addresses to local addresses. Using exact data-flow information, our compiler is capable of more aggressive communication optimizations than compilers that rely on data dependence analysis. Our code generation algorithms can also be used to enhance the memory system performance of machines with a shared address space.

2 Array Data-Flow Analysis

Previous research on parallelizing compilers is based primarily on data dependence analysis. The domain of data dependence analysis is that the array index function and the loop bounds must be integer affine functions of loop indices and possibly other symbolic constants. The traditional formulation of the data dependence problem can best be described as memory disambiguation. That is, data dependence checks if *any* of the dynamic instances of two array accesses in a loop nest refers to the same *location*. Consider the following example:

```

for  $i := 0$  to  $N$  do
  for  $j := 3$  to  $N$  do
     $X[j] := X[j - 3];$ 

```

Data dependence analysis of this program will produce the dependence vectors $\{[+, 3], [0, 3]\}$, meaning that the read access in iteration $[i_r, j_r]$ may be data dependent on all iterations $[i_w, j_w]$ such that $j_w = j_r - 3$ and $i_w \leq i_r$. There are two sources of inaccuracy in this result. First, data dependence does not contain *coverage* information. For this example, the fact that the data *value* used by the read is produced in the same i iteration ($i_w = i_r$) cannot be found using data dependence analysis. Second, data dependence information lacks the *identity* of the dependence. That is, the fact that the first three iterations of i use data defined outside the loop nest is not captured by the data dependence analysis. Unlike data dependence analysis, exact array data-flow analysis determines if two dynamic instances refer to the same value, and not just to the same location. This information is found for every dynamic read-write access pair. Thus, exact data-flow analysis does not have the inaccuracies of the traditional data dependence analysis.

The problem of finding exact array data-flow information was first formulated by Feautrier[4, 5, 6]. Feautrier proposed a parametric integer programming algorithm that can find such exact information in the domain of loop nests where the loop bounds and array indices are affine functions of loop indices. We have developed an algorithm that can find the same exact information for many common cases more efficiently. The data-flow information generated by our array data-flow analysis is captured by a representation called a *Last Write Tree* (LWT)[13, 14]. The LWT is a function that maps an instance of a read operation to the very write instance that produces the value read, provided such a write exists. We denote a read and a write instance by the values of their loop indices, $\mathbf{i}_r$ and $\mathbf{i}_w$, respectively. The domain of the LWT function is the set of read iterations $\mathbf{i}_r$ that satisfy the constraints imposed by the loop bounds. Each internal node in the tree contains a further constraint on the value of the read instance $\mathbf{i}_r$. It partitions the domain of read instances into those satisfying the constraint, represented by its right descendants, and those not satisfying the constraint, represented by its left descendants. We define the *context* of a node to be the set of read iterations that satisfy the constraints of a node’s ancestors. The tree is constructed such that either all the values read within the context of a leaf are written within the loop, or none of them are. In the former case, the leaf node defines a *last-write relation* that relates each read instance $\mathbf{i}_r$ within the context to the write instance $\mathbf{i}_w$ that produces the value read. All the read-write pairs share the same dependence levels. If the instances within a context do not read any value written within the loop, we denote the lack of a writer by $\perp$.

As another example, consider the code for a single stage of an Alternating Direction Implicit (ADI) integration shown in Figure 1.

```

(1) for  $i_1 := 1$  to  $N$  do
    for  $i_2 := 0$  to  $N$  do
         $X[i_1, i_2] := f_1(X[i_1, i_2], X[i_1 - 1, i_2], Y[i_1, i_2]);$ 

(2) for  $i_1 := 0$  to  $N$  do
    for  $i_2 := 1$  to  $N$  do
         $X[i_1, i_2] := f_2(X[i_1, i_2], X[i_1, i_2 - 1], Y[i_1, i_2]);$ 

```

Fig. 1. ADI integration code.

We will use this code as a example throughout the paper. Loop nest 1 sweeps along the first dimension of the array X and loop nest 2 sweeps along the second dimension. This kernel is representative of the computation used in the heat conduction phase of the benchmark SIMPLE from Lawrence Livermore National Laboratory. The LWT information for the read access $X[i_1 - 1, i_2]$ with respect to the write access $X[i_1, i_2]$ of loop nest 1 of the example is given in Figure 2. Let (i_{1r}, i_{2r}) be the loop index values of an instance of a read access. The root node

represents the domain of the read iterations. The single internal node divides the read iterations into two sections. If $i_{1r} \geq 2$ then the read iterations use a value produced by the instance (i_{1w}, i_{2w}) of the write access, where $i_{1w} = i_{1r} - 1$ and $i_{2w} = i_{2r}$. This is represented by the right leaf. If $i_{1r} < 2$ then the value read is defined outside the loop nest and is denoted by $\perp$ in the left leaf.

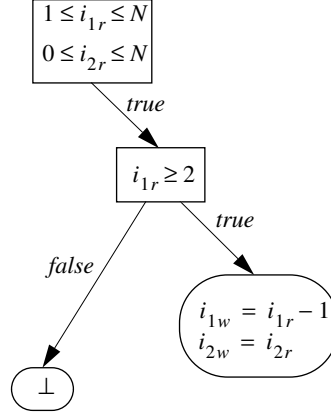


Fig. 2. The Last Write Tree for the read access $X[i_1 - 1, i_2]$ with respect to the write access $X[i_1, i_2]$ in loop nest 1.

We have developed an array privatization algorithm based on the LWT analysis[14]. The algorithm first parallelizes the loops by considering only the *data-flow* dependence constraints. If the parallelized loop carries any anti or output dependences, privatization is necessary. On machines with a shared address space, the compiler needs to create a private copy of the data for every processor. The private copies may need to be initialized and the final results may need to be written back to the original array. On machines with a distributed address space, where every processor must keep a local copy of the data accessed, privatized arrays are treated no differently from any other data. The compiler will not generate any communication unless there is a data-flow dependence.

In addition to enabling optimizations such as array privatization, LWTs are also useful for generating efficient code for distributed address space machines, as discussed in Section 4.

3 Data and Computation Decompositions

This section describes our compiler algorithm to find data and computation decompositions that maximize parallelism and minimize communication. First a local, loop-level analysis phase transforms the code to expose the maximum degree of parallelism. Then a global analysis determines the best mapping of data and computation across the processors of the machine.

3.1 Loop Transformations for Maximizing Parallelism

The compiler first transforms the code to expose the maximum degree of loop-level parallelism, while minimizing the frequency of communication. It tries to generate the coarsest granularity of parallelism by placing the largest degree of parallelism in the outermost positions of the loop nest. Since no communication is needed between iterations of a parallel loop, pushing the parallel iterations outermost reduces the frequency of communication.

The scope of our loop transformations is interchange, skewing, reversal and tiling. All but the tiling transformations are unified as unimodular matrix transforms on the iteration space representing the computation. The problem thus reduces to finding the best combination of a unimodular matrix transform and tiling that yields the maximum degree of coarse-grain parallelism, while satisfying the data flow or data dependence constraints.

Choices of Parallelism. Our algorithm for maximizing loop-level parallelism is based on the concept of *fully permutable* loop nests. A loop nest is fully permutable if any arbitrary permutation of the loops within the nest is legal[10, 19]. Fully permutable loop nests can be easily transformed to contain coarse-grain parallelism. In particular, every fully permutable loop nest of depth k has at least $k - 1$ degrees of parallelism. In the degenerate case where the loop nest has no loop-carried dependences, it has k degrees of parallelism.

A loop nest with only distance vectors has the property that it can always be transformed into a fully permutable loop nest. Loop nests with direction vectors, however, cannot always be converted into a single fully permutable loop nest. The compiler uses unimodular transformations to transform the code into the canonical format of a nest of fully permutable subnests, such that each subnest is made as large as possible starting with the outermost loops. Within each subnest, loops that are already *doall* loops in the source program are placed in the outermost positions. A *doall* loop has no loop-carried dependences and can thus execute in parallel with no communication. The maximum degree of (outermost) parallelism for the entire loop nest is simply the sum of the degree of parallelism contained in each of the fully permutable subnests.

For the example from Figure 1, the compiler interchanges the loops in the first loop nest to produce the code shown in Figure 3. The outer loops are *doall* loops, and both loop nests are fully permutable. The resulting dependence vectors are shown to the right of each loop nest.

The parallelism in a fully permutable loop nest can be exploited in many ways. Consider the first loop nest in the ADI example. Since the *doall* (outer) loop accesses columns of arrays X and Y , iterations of the *doall* loop can run in parallel with no communication if columns of the array are distributed across the processors. The original 2-D iteration space is shown in Figure 4(a) and Figure 4(b) shows the 1-D parallel execution of the *doall* loop.

When iterations of a loop with loop-carried dependences are allocated to different processors, explicit synchronization and communication are required

```

(1) for  $i_2 := 0$  to  $N$  do /* doall */
    for  $i_1 := 1$  to  $N$  do
         $X[i_1, i_2] := f_1(X[i_1, i_2], X[i_1 - 1, i_2], Y[i_1, i_2]);$     Dependences =  $\{(0, 1)\}$ 

(2) for  $i_1 := 0$  to  $N$  do /* doall */
    for  $i_2 := 1$  to  $N$  do
         $X[i_1, i_2] := f_2(X[i_1, i_2], X[i_1, i_2 - 1], Y[i_1, i_2]);$     Dependences =  $\{(0, 1)\}$ 

```

Fig. 3. ADI integration code, after parallelization.

within the computation of the loop. Opportunities for *doacross* parallelism occur when a fully permutable loop nest contains at least two loops. *Doacross* parallelism is also available in the first loop nest of the ADI code. The loop with the loop-carried dependence (the inner loop) accesses rows of the arrays. For example, if each processor calculates an iteration of this loop, and rows of the arrays are distributed across the processors, then point-to-point communication is required between neighboring processors. Figure 4(c) shows the 1-D parallel execution of the loop nest using *doacross* parallelism.

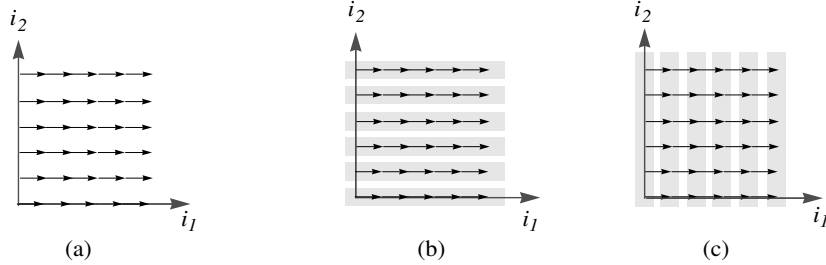


Fig. 4. (a) Original iteration space. (b)-(c) Iteration spaces showing parallel execution of the first loop nest in the ADI example. The arrows represent data dependencies, and the iterations in each shaded region are assigned to the same processor.

In Figure 4(b) we allocated iterations along the direction $(1, 0)$ to each processor, whereas in Figure 4(c) we allocated iterations along the direction $(0, 1)$. In fact, it is possible to exploit parallelism by allocating to each processor iterations along any direction within these two axes.

It is possible to reduce the communication volume and frequency by tiling (also known as blocking, unroll-and-jam and stripmine-and-interchange)[18, 20]. In general, tiling transforms a loop nest of depth k into a loop nest of depth $2k$. The inner k loops iterate over a fixed number of iterations, while the outer loops

iterate across the inner blocks of iterations. By tiling and then parallelizing only the outer loops, the synchronization frequency and communication volume are reduced by the size of the block. A fully permutable loop nest has the property that it can be completely tiled. Since the compiler first transforms the loop nests into the canonical form of nests of fully permutable subnests, the tiling transformation is easily applied.

Global Considerations. If we look at each loop nest individually, then distributing the iterations in the direction of the *doall* loops is preferable, as no communication is necessary. However, this is not always the case when we analyze multiple loop nests together. In the ADI example, consider what happens if we only try to exploit the parallelism in the *doall* loops. The *doall* loop in the first loop nest accesses columns of arrays X and Y , whereas iterations of the *doall* loop in the second loop nest accesses rows of the arrays. Communication will occur between the loop nests because the data must be completely reorganized as the data distribution switches between rows and columns.

We can avoid redistributing the arrays between the first and second loop nest by using *doacross* parallelism in one of the loop nests. For example, in the second loop nest the loop with the loop-carried dependence (the inner loop), accesses columns of the array. If iterations of this loop are distributed across the processors, then point-to-point communication is required between neighboring processors. However, no data reorganization is needed between the two loop nests, and the point-to-point communication is typically a less expensive form of communication.

The original iteration space for loop nests 1 and 2 in the ADI example from Figure 3 is shown in Figure 5(a). The iteration space looks the same for both loop nests because the code has been transformed so that the parallel loops are outermost. Figure 5(b) and 5(c) illustrate how the loop nests can be executed in parallel (tiling has been applied to both loop nests). Each processor is assigned a block of array columns and the corresponding strip of the iteration space. In loop nest 1, the data dependences are within the blocks so no communication is necessary. In loop nest 2, there are dependences across the blocks and explicit synchronization is used to enforce the dependences between the blocks.

As this example illustrates, only exploiting the parallelism in the *doall* loops may not result in the best overall performance. In general, there may be tradeoffs between the best loop-level decompositions, and the best global decompositions. Thus the loop-level analysis in our compiler transforms the code to expose the maximum degree of parallelism, but does not make decisions as to how that parallelism is to be implemented. The loop-level analysis leaves the code in a canonical format of fully permutable loop nests, from which the coarsest degree of parallelism can be easily derived.

3.2 Maximizing Locality

Effective utilization of a machine’s memory hierarchy is important for achieving high performance. Since interprocessor communication is the most expensive

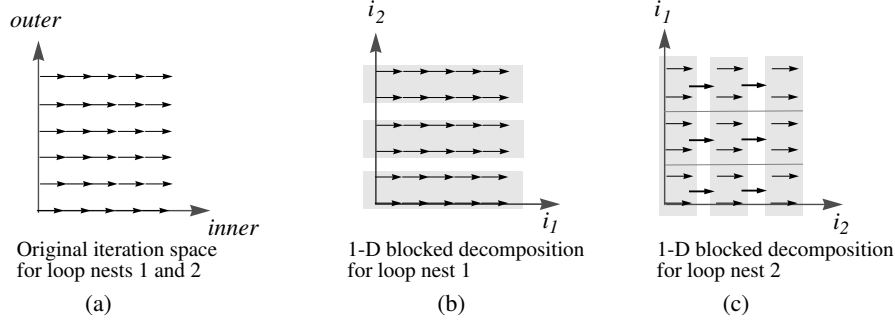


Fig. 5. (a) Original iteration space. (b)-(c) Iteration spaces showing the parallel execution of tiled loops for the ADI example. The arrows represent data dependencies.

form of data movement in the memory hierarchy of a multiprocessor, minimizing such communication is therefore the most critical locality optimization. This is achieved by transforming the code to maximize the coarsest granularity of parallelism. The next step is to optimize for the memory hierarchies in a single processor, such as caches and registers. We have developed a data locality optimizing algorithm within our unified loop transformation framework[18]. We apply this uniprocessor algorithm to the subnest consisting of the distributed parallel loops and their inner loops. The original loop structure differs from this subnest by having additional sequential loops outside the parallel loops. Since these sequential loops must be placed outermost due to legality reasons, the uniprocessor data locality is not compromised by parallelization.

3.3 Finding Decompositions

We model decompositions as having two components. The computation and data are first mapped onto a virtual processor space using affine functions. The virtual processor space has as many processors as is needed to fit the number of loop iterations and the sizes of the arrays. Next a folding function is used to map the virtual processor space onto the physical processors of the target machine. The physical processor mapping takes load-balance considerations into account, and reduces the communication cost by placing blocks of communicating virtual processors onto the same physical processor. This two-step model is able to capture a wide range of computation and data assignments. For example, our model represents a superset of the decompositions available to HPF[7] programmers.

Virtual Processor Mapping. The objective of the virtual processor mapping phase is to minimize communication while maintaining sufficient parallelism. This phase eliminates communication in two ways. First, it explicitly chooses which loops to distribute across the processors – it will consider trading off excess parallelism to reduce or eliminate communication. Also, for a given degree

of parallelism, it aligns the arrays and loop iterations to reduce nonlocal data accesses.

Modeling Decompositions. In this discussion, all loops are normalized to have a unit step size, and all arrays subscripts are adjusted to start at 0. A loop nest of depth l defines an iteration space $\mathcal{I}$. Each iteration of the loop nest is identified by its index vector $\mathbf{i} = (i_1, i_2, \dots, i_l)$. An array of dimension m defines an array space $\mathcal{A}$, and each element in the array is accessed by an integer vector $\mathbf{a} = (a_1, a_2, \dots, a_m)$. Similarly, an n -dimensional processor array defines a processor space $\mathcal{P}$. We write an affine array index function $\mathbf{f} : \mathcal{I} \rightarrow \mathcal{A}$ as $\mathbf{f}(\mathbf{i}) = F\mathbf{i} + \mathbf{k}$, where F is a linear transformation and $\mathbf{k}$ is a constant vector.

Definition 1. For each index $\mathbf{a}$ of an m -dimensional array, the *data decomposition* of the array onto an n -dimensional processor array is a function $\delta(\mathbf{a}) : \mathcal{A} \rightarrow \mathcal{P}$, where

$$\delta(\mathbf{a}) = D\mathbf{a} + \mathbf{d}$$

D is an $n \times m$ linear transformation matrix and $\mathbf{d}$ is a constant vector.

Definition 2. For each iteration $\mathbf{i}$ of a loop nest of depth l , the *computation decomposition* of the loop nest onto an n -dimensional processor array is a function $\gamma(\mathbf{i}) : \mathcal{I} \rightarrow \mathcal{P}$, where

$$\gamma(\mathbf{i}) = C\mathbf{i} + \mathbf{c}$$

C is an $n \times l$ linear transformation matrix and $\mathbf{c}$ is a constant vector.

Let the computation decomposition for loop nest j be $\gamma_j = C_j(\mathbf{i}) + \mathbf{c}_j$ and the data decomposition for array x in loop j be $\delta_{xj} = D_{xj}(\mathbf{a}) + \mathbf{d}_{xj}$. Furthermore, let $\mathbf{f}_{xj}$ be an array index function for array x in loop nest j . No communication is required if it is possible to define a computation decomposition for each loop nest j and a data decomposition for each array x such that the following equation holds for all iterations $\mathbf{i}$:

$$D_{xj}(\mathbf{f}_{xj}(\mathbf{i})) + \mathbf{d}_{xj} = C_j(\mathbf{i}) + \mathbf{c}_j \quad (1)$$

If it is possible to find a single non-trivial, decomposition with no communication, then there exist many equivalent decompositions with the same degree of parallelism and no communication. For example, given a communication-free decomposition we could always transpose all the data and computation and still have no communication. We make the observation that the property shared by equivalent communication-free decompositions is the data and computation that is assigned to the same processor. We use this observation to reduce the complexity of finding the decomposition functions. Our approach is to first solve for the data and computation that are mapped onto the same processor. A simple calculation can then be used to find an assignment of data and computation to specific processors[2].

The data decompositions $\delta(\mathbf{a}) = D\mathbf{a} + \mathbf{d}$ and computation decompositions $\gamma(\mathbf{i}) = C\mathbf{i} + \mathbf{c}$ have two components. The linear transformation matrices D and

C describe the mapping between the axes of the array elements and loop iterations, and the processors. The constant vectors $\mathbf{d}$ and $\mathbf{c}$ give the displacement of the starting position of the data and computation. Communication due to mismatches in the linear transformation component of a decomposition is considered expensive, since it requires a reorganization of the entire array. In contrast, communication at the displacement level is inexpensive since the amount of data transferred is significantly reduced by blocking. Thus the priority is in finding the best linear transformation, and we focus on the version of Eqn. 1 that omits displacements. Letting $\mathbf{f}_{xj}(\mathbf{i}) = F_{xj}(\mathbf{i}) + \mathbf{k}_{xj}$,

$$D_x F_{xj}(\mathbf{i}) = C_j(\mathbf{i}) \quad (2)$$

Algorithm Overview. This section presents a brief overview of the decomposition algorithms we have developed. The details of the algorithms can be found in a related paper[2].

Each loop nest has been transformed into canonical form to expose the maximum degree of coarse-grain parallelism. The loop nests with parallelism may be nested within outer sequential loops. The outer sequential loops are considered in order from innermost to outermost. At each sequential loop level, the decomposition algorithm tries to eliminate communication by aligning the data and computation and also by reducing the degree of parallelism where necessary. If it cannot eliminate communication while still maintaining at least one degree of parallelism, then communication is necessary.

Finding decompositions that minimize communication while allowing at least one degree of parallelism is NP-hard[2]. We thus use a greedy algorithm to eliminate the largest amounts of potential communication first. To model the potential communication in a program, the compiler uses a *communication graph*. The nodes in the graph correspond to the loop nests with parallelism in the program, and the edges are labeled with worst-case estimates of the communication cost between the nodes.

We use a greedy algorithm that examines the edges in order of decreasing weights. For each edge, it looks at the two loop nests connected by the edge. The algorithm first tries to find a decomposition that only distributes the completely parallel, or *doall*, loops of the two loop nests, such that there is no data reorganization communication. If such a decomposition cannot be found that still has at least one degree of parallelism, the algorithm then expands the search to also consider *doacross* parallelism. Finally, if communication is required between the loop nests, the algorithm tries to find a decomposition with the minimum communication volume across the two loops nests.

We want to avoid making unnecessary choices in our greedy algorithm. If there is no non-displacement communication, then equivalent decompositions have the same data and computation assigned to a single processor. Thus each step of the greedy algorithm first collects constraints that specify which array elements and loop iterations must be mapped onto the same processor. These constraints are represented in terms of constraints on each individual array and loop nest with respect to itself. Mathematically, the constraints form the nullspace

of the linear transformation matrices D and C for each array and loop nest, respectively.

After the nullspaces have been found, the full linear transformation matrices (D and C) and displacements ($\mathbf{d}$ and $\mathbf{c}$) are then calculated to specify the complete virtual processor mapping. If there exist non-trivial decompositions that do not require any communication beyond the displacement level, our algorithm is optimal in that it will find the decomposition with the maximum degree of parallelism[2]. Once the algorithm finds the decompositions for a single instance of the current enclosing sequential loop, it then finds decompositions for all instances of that outer loop using the same greedy strategy.

Consider the ADI example from Figure 3. The algorithm first tries to find decompositions for loop nests 1 and 2 with no data reorganization communication using only the *doall* loops. Since the only such decomposition is to run the loops sequentially by placing all the data and computation on a single processor, the algorithm next considers using *doacross* parallelism. The loop nests in the example are fully permutable, so the algorithm then considers all the loops as candidates for distribution. The resulting computation decompositions the compiler finds (shown in Figures 5(b) and 5(c)) are $C_1 = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} i_2 \\ i_1 \end{bmatrix}$ and $C_2 = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} i_1 \\ i_2 \end{bmatrix}$. The corresponding data decompositions are $D_X = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$ and $D_Y = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$.

Physical Processor Mapping. After finding the affine virtual processor mapping, the compiler then determines the physical processor mapping. The goal of this step is to effectively utilize the limited physical resources and to further optimize communication for the target architecture.

Currently, our compiler only considers three discrete mapping functions for each distributed dimension; they correspond to the choices of “block”, “cyclic” and “block-cyclic” in the HPF terminology. It is possible to specify the mapping function by simply specifying the block size. Let N be the number of loop iterations and P is the number of processors, the block sizes for the three mappings are, respectively, $\lceil \frac{N}{P} \rceil$, 1, and b where $1 < b < \lceil \frac{N}{P} \rceil$ and b is determined based on the performance parameters of the machine. (N and P need not be compile-time constants in the program).

The choice of physical mapping is influenced by several factors. First, the affine decomposition phase might decide to block a loop (and arrays used in the loop) to exploit *doacross* parallelism efficiently. Second, if the execution time of each iteration in a parallel loop is highly variable, a cyclic mapping is needed to obtain good load balance. An example of such code is if the parallel loop contains an inner loop whose iteration count is a function of the index of the parallel loop. Finally, to reduce communication, it is desirable to use the same data mapping for the same array across different loops. The compiler tries to satisfy all these constraints using an iterative algorithm similar to the affine

decomposition algorithm. For example, if an array has a block constraint for one loop nest, and a cyclic constraint in another loop nest, then the compiler will choose a block-cyclic physical mapping. When no constraint is imposed on a loop, the default is to distribute by blocks of size $\lceil \frac{N}{P} \rceil$. For the ADI example, all the distributed loops and arrays are distributed by blocks of size $b = \lceil \frac{N}{P} \rceil$.

4 Code Generation

After finding the decompositions, the next phase in our compiler is to transform and optimize the code allotted to each processor. For a machine with a shared address space, this phase is straightforward. However, if the machine has a distributed address space, the compiler must also manage the memory and communication explicitly. This section describes how to generate correct and efficient code for distributed address space machines.

4.1 When and Where to Communicate

While exact data-flow information is useful in determining the necessary communication, the information may not be available due to, for example, non-affine loop bounds or conditional control flow. Our algorithm uses data-flow information whenever it is available.

Our compiler preprocesses the program and identifies those contiguous regions of code where exact data-flow information is available. It further divides the rest of the program into regions, such that the code with different data decompositions belongs to different regions. Thus, a region either has a uniform data decomposition, or exact data-flow information is available. There are two forms of communication: communication at the boundaries of different regions, and communication within a region. Massive data movements between regions can be optimized by mapping them to hand-tuned collective communication routines.

If data-flow information is not available, we plan to use an algorithm similar to that used in FORTRAN D to generate communication [16]. In our case, however, both the computation and data decomposition are specified by our compiler. Communication is necessary whenever the processor executing an operation does not own the data used. The compiler uses data dependence analysis to perform communication optimizations such as message aggregation.

If data-flow information is available, our communication generation algorithm uses the given data decompositions only as boundary conditions for the region[1]. This allows for more flexible data decompositions within a region including data replication and data migration across different processors. Communication within the region is captured by all the non- $\perp$ contexts in the LWT generated by our data-flow analysis algorithm. The LWT specifies the exact write instance that produces the *value* used by each read instance. Communication is necessary if the producer and consumer instances are to be executed by different processors. The LWT groups the read instances into contexts, such that all

the instances within a context have the same distance or direction vector. This structure facilitates optimizations such as message aggregation and hiding the latency of communication with communication.

4.2 Communication Code Generation

We have developed a uniform mathematical framework to solve the problems of communication code generation[1]. We represent data decompositions, computation decompositions and the array access information all as systems of linear inequalities.

For the ADI example from Figure 3, the earlier compiler phase decides to block the second dimension of both arrays X and Y (Section 3.3). That is, the array element $[a_1, a_2]$ resides in processor p if and only if $bp \leq \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} < bp + b$, where b is the block size. The computation decomposition decision is to block the outer loop in the first loop nest and to block the inner loop in the second loop nest. Thus, iteration (i_2, i_1) in the first loop nest executes on processor p if and only if $bp \leq \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} i_2 \\ i_1 \end{bmatrix} < bp + b$, where b is the block size. Similarly, iteration (i_1, i_2) in the second loop nest executes on processor p if and only if $bp \leq \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} < bp + b$, where b is the block size.

We also represent a set of communication by a system of linear inequalities. We derive the receiving and sending code for each communication set by projecting the polyhedra represented by the system of linear inequalities onto lower-dimensional spaces in different orders[1].

if $p \geq 0$ and $p \leq \lfloor N/b \rfloor$ then for $i_1 := 0$ to $\lfloor N/b \rfloor$ do for $i_2 := \max(1, bp)$ to $\min(b(p+1) - 1, N)$ do $X[i_1, i_2] := f(X[i_1, i_2], X[i_1, i_2 - 1]);$	
(a) Computation loop nest.	
if $p_s \geq 0$ and $p_s \leq \lfloor N/b \rfloor - 1$ then for $i_{1s} := 0$ to $\lfloor N/b \rfloor$ do $i_{2s} := b(p_s + 1) - 1;$ $p_r := p_s + 1;$ $i_{1r} := i_{1s};$ $i_{2r} := bp_r;$ $a_1 := i_{1s};$ $a_2 := i_{2s};$ Send $X[a_1, a_2]$ to processor p_r;	if $p_r \geq 1$ and $p_r \leq \lfloor N/b \rfloor$ then for $i_{1r} := 0$ to $\lfloor N/b \rfloor$ do $i_{2r} := bp_r;$ $p_s := p_r - 1;$ $i_{1s} := i_{1r};$ $i_{2s} := b(p_s + 1) - 1;$ $a_1 := i_{1r};$ $a_2 := i_{2r} - 1;$ Receive $X[a_1, a_2]$ from processor p_s;
(b) Send loop nest.	(c) Receive loop nest.

Fig. 6. Loop nests generated for computation and communication.

In the ADI example, communication is necessary only for the second loop nest and only for the array X . Figure 6 shows the three components of code that each processor has to execute in the second loop. We concentrate on array X as it is the only array that needs communication. These components are merged together to form a single SPMD program after optimizations.

4.3 Communication Optimization

Our compiler performs several communication optimizations. These optimizations include eliminating redundant messages, aggregating messages, and hiding communication latency by overlapping the communication and computation. The data-flow information enables the compiler to perform these optimizations more aggressively than would otherwise be possible using data dependence information[1].

The compiler uses the LWT information to eliminate redundant data transfers. While accessing the same location may require multiple data transfers since the value at the location may or may not have changed, each value needs to be transferred once and only once.

The perfect producer and consumer information enables the compiler to issue the send immediately after the data are produced and to issue the receive just before the data are used. This maximizes the chances that the communication is overlapped with computation. However, there is a tradeoff between this optimization and message aggregation. Maximum overlapping of communication and computation generates a large number of messages, creating a very high message overhead.

Thus, based on estimates using the performance parameters of the architecture, the compiler may create a single message for a block of iterations to achieve a balance between parallelism and communication overhead. For the ADI example, the outer loop is blocked and messages are aggregated within each block, as shown in Figure 7.

4.4 Local Address Space

Typically, a processor on a parallel machine touches only a part of an array. Since data sets processed by these programs are often very large, it is essential that the compiler only allocates, on each processor, storage for the data used by that processor.

The following is a simple approach to the memory allocation problem. We allocate on each processor the smallest rectangular region covering all the data read or written by the processor, and we copy all the received data from the communication buffer to their home locations in the local array before they are accessed. If there are multiple accesses to the same array, we simply find the bounding box of the rectangular boxes for all the accesses to the same array. Note that this formulation allows local data spaces on different processors to overlap. The computation loop nest with the local address of X is given in Figure 8.

if $p \geq 0$ and $p \leq \lfloor N/b \rfloor$ then for $i'_1 := 0$ to $\lfloor N/t \rfloor$ do for $i_1 := ti'_1$ to $\min(t(i'_1 + 1) - 1, N)$ do for $i_2 := \max(1, bp)$ to $\min(b(p + 1) - 1, N)$ do $X[i_1, i_2] := f(X[i_1, i_2], X[i_1, i_2 - 1]);$
(a) Computation loop nest.
if $p \geq 0$ and $p \leq \lfloor N/b \rfloor - 1$ then for $i'_1 := 0$ to $\lfloor N/t \rfloor$ do $p_r := p + 1;$ initialize(msg); for $i_{1s} := ti'_1$ to $\min(t(i'_1 + 1) - 1, N)$ do $i_{2s} := b(p + 1) - 1;$ enqueue(msg, $X[i_{1s}, i_{2s}]$); Send msg to processor p_r ;
(b) Send loop nest.
if $p \geq 1$ and $p \leq \lfloor N/b \rfloor$ then for $i'_1 := 0$ to $\lfloor N/t \rfloor$ do $p_s := p - 1;$ Receive msg from processor p_s ; for $i_{1r} := ti'_1$ to $\min(t(i'_1 + 1) - 1, N)$ do $i_{2r} := bp;$ $X[i_{1r}, i_{2r} - 1] = \text{dequeue}(\text{msg});$
(c) Receive loop nest.

Fig. 7. Blocking the outer loop for message aggregation with block size t .

```

double  $X[N, b + 1];$ 
if  $p \geq 0$  and  $p \leq \lfloor N/b \rfloor$  then
    for  $i'_1 := 0$  to  $\lfloor N/t \rfloor$  do
        for  $i_1 := ti'_1$  to  $\min(t(i'_1 + 1) - 1, N)$  do
            for  $i_2 := \max(1, bp)$  to  $\min(b(p + 1) - 1, N)$  do
                 $X[i_1, i_2 - (bp - 1)] = f(X[i_1, i_2], X[i_1, i_2 - 1 - (bp - 1)]);$ 

```

Fig. 8. Local address space mapping (from Figure 7(a)).

Further optimizations to reduce the local address space are possible if data-flow information is available. A processor can operate on the communication buffer directly, and it is not necessary to find the smallest rectangular box covering the data used throughout the entire program.

4.5 Merging Loop Nests

Our code generation algorithm creates separate code for each of the three components of an SPMD program: the computation, the receiving side of the communication and the sending side of the communication. Each of these components specify the actions to be performed in a subset of the source iterations. The

final step is thus to merge these components together so that the operations are performed at the correct time[1]. The final SPMD program for our ADI example is given in Figure 9.

5 Conclusion

In this paper, we showed how the complex problem of compiling for distributed memory machines can be divided into several well-defined subproblems: array data flow analysis, data and computation decomposition and code generation.

Compilers for distributed memory machines require the more sophisticated analysis of exact data-flow analysis on individual array elements. This information allows the compiler to perform more advanced optimizations than with traditional data dependence analysis. In particular, our compiler uses array data-flow information to perform array privatization, as well as communication code generation and optimization.

The parallelism and communication in a program are inter-related; opportunities for parallelism must be weighed against the resulting communication cost. The parallelization phase of the compiler transforms the code to expose the maximum degree of parallelism and minimize the frequency of communication for each loop nest. The decomposition phase then selects among the degrees of parallelism to find computation and data decompositions that reduce communication.

For the given decompositions, the final phase of the compiler generates an SPMD program. By using exact data-flow information, our communication code generation algorithm gains several advantages over the traditional approach based on data dependence. First, it supports more flexible data decomposition schemes; data do not require a static home, data may be replicated, and the owner-computes rule can be relaxed. Code generation also involves allocating local space on each processor for distributed data, translating the global data addresses to local addresses and generating all the necessary receive and send instructions.

We model each stage of the compilation mathematically. The parallelization phase uses a framework that unifies transformations such as loop skewing, reversal and permutation by representing them as unimodular matrix transformations. The decompositions are represented as the composition of two functions: an affine function mapping the computation and data onto a virtual processor space, and a folding function mapping the virtual space onto physical processors of the target machine. By using this model, our algorithm avoids expensive searches through the program transformation space by solving for these mappings directly. The code generation algorithms represent communication as systems of linear inequalities. The various code generation and communication optimization problems are solved within this framework.

The next step of this research is to run experiments with real programs. We need to gain experience with the compiler system, and to test its effectiveness and practicality. In the future, we plan to extend the parallelization phase to

```

double  $X[N, b + 1]$ ;

if  $p = 0$  then
  for  $i'_1 := 0$  to  $\lfloor N/t \rfloor$  do
    /* Computation */
    for  $i_1 := ti'_1$  to  $\min(t(i'_1 + 1) - 1, N)$  do
      for  $i_2 := \max(1, bp)$  to  $\min(b(p + 1) - 1, N)$  do
         $X[i_1 - (bp - 1), i_2 - (bp - 1)] := f(X[i_1 - (bp - 1), i_2 - (bp - 1)],$ 
           $X[i_1 - (bp - 1), i_2 - 1 - (bp - 1)]);$ 

      /* Send */
       $p_r := p + 1;$ 
      initialize(msg);
      for  $i_{1s} := ti'_1$  to  $\min(t(i'_1 + 1) - 1, N)$  do
         $i_{2s} := b(p + 1) - 1;$ 
        enqueue(msg,  $X[i_{1s} - (bp - 1), i_{2s} - (bp - 1)]$ );
      Send msg to processor  $p_r$ ;

if  $p \geq 1$  and  $p \leq \lfloor N/b \rfloor - 1$  then
  for  $i'_1 := 0$  to  $\lfloor N/t \rfloor$  do
    /* Receive */
     $p_s := p - 1;$ 
    Receive msg from processor  $p_s$ ;
    for  $i_{1r} := ti'_1$  to  $\min(t(i'_1 + 1) - 1, N)$  do
       $i_{2r} := bp;$ 
       $X[i_{1r} - (bp - 1), i_{2r} - 1 - (bp - 1)] = \text{dequeue}(\text{msg});$ 
    /* Computation */
    for  $i_1 := ti'_1$  to  $\min(t(i'_1 + 1) - 1, N)$  do
      for  $i_2 := \max(1, bp)$  to  $\min(b(p + 1) - 1, N)$  do
         $X[i_1 - (bp - 1), i_2 - (bp - 1)] := f(X[i_1 - (bp - 1), i_2 - (bp - 1)],$ 
           $X[i_1 - (bp - 1), i_2 - 1 - (bp - 1)]);$ 

      /* Send */
       $p_r := p + 1;$ 
      initialize(msg);
      for  $i_{1s} := ti'_1$  to  $\min(t(i'_1 + 1) - 1, N)$  do
         $i_{2s} := b(p + 1) - 1;$ 
        enqueue(msg,  $X[i_{1s} - (bp - 1), i_{2s} - (bp - 1)]$ );
      Send msg to processor  $p_r$ ;

if  $p = \lfloor N/b \rfloor$  then
  for  $i'_1 := 0$  to  $\lfloor N/t \rfloor$  do
    /* Receive */
     $p_s := p - 1;$ 
    Receive msg from processor  $p_s$ ;
    for  $i_{1r} := ti'_1$  to  $\min(t(i'_1 + 1) - 1, N)$  do
       $i_{2r} := bp;$ 
       $X[i_{1r} - (bp - 1), i_{2r} - 1 - (bp - 1)] = \text{dequeue}(\text{msg});$ 
    /* Computation */
    for  $i_1 := ti'_1$  to  $\min(t(i'_1 + 1) - 1, N)$  do
      for  $i_2 := \max(1, bp)$  to  $\min(b(p + 1) - 1, N)$  do
         $X[i_1 - (bp - 1), i_2 - (bp - 1)] := f(X[i_1 - (bp - 1), i_2 - (bp - 1)],$ 
           $X[i_1 - (bp - 1), i_2 - 1 - (bp - 1)]);$ 

```

Fig. 9. The SPMD code for the example. 18

perform transformations such as loop reindexing, fission and fusion to expose greater degrees of parallelism. Currently, the compiler operates on each procedure individually. We plan to extend our algorithms to do interprocedural analysis.

References

1. S. P. Amarasinghe and M. S. Lam. Communication optimization and code generation for distributed memory machines. In *Proceedings of the SIGPLAN '93 Conference on Programming Language Design and Implementation*, June 1993.
2. J. M. Anderson and M. S. Lam. Global optimizations for parallelism and locality on scalable parallel machines. In *Proceedings of the SIGPLAN '93 Conference on Programming Language Design and Implementation*, June 1993.
3. B. Chapman, P. Mehrotra, and H. Zima. Programming in Vienna Fortran. *Scientific Programming*, 1(1):31–50, Fall 1992.
4. P. Feautrier. Parametric integer programming. Technical Report 209, Laboratoire Methodologie and Architecture Des Systemes Informatiques, January 1988.
5. Paul Feautrier. Array expansion. In *International Conference on Supercomputing*, pages 429–442, July 1988.
6. Paul Feautrier. Dataflow analysis of scalar and array references. *Journal of Parallel and Distributed Computing*, 20(1):23–53, February 1991.
7. High Performance Fortran Forum. *High Performance Fortran Language Specification*, January 1993. Draft Version 1.0.
8. S. Hiranandani, K. Kennedy, and C.-W. Tseng. Compiling Fortran D for MIMD distributed-memory machines. *Communications of the ACM*, 35(8):66–80, August 1992.
9. Intel Corporation, Santa Clara, CA. *iPSC/2 and iPSC/860 User's Guide*, June 1990.
10. F. Irigoien and R. Triolet. Supernode partitioning. In *Proceedings of the SIGPLAN '88 Conference on Programming Language Design and Implementation*, pages 319–329, January 1988.
11. C. Koelbel, P. Mehrotra, and J. Van Rosendale. Supporting shared data structures on distributed memory architectures. In *Proceedings of the Second ACM/SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 177–186, March 1990.
12. D. Lenoski, K. Gharachorloo, J. Laudon, A. Gupta, J. Hennessy, M. Horowitz, and M. Lam. The Stanford DASH Multiprocessor. *IEEE Computer*, 25(3):63–79, March 1992.
13. D. E. Maydan. *Accurate Analysis of Array References*. PhD thesis, Stanford University, September 1992. Published as CSL-TR-92-547.
14. D. E. Maydan, S. P. Amarasinghe, and M. S. Lam. Array data flow analysis and its use in array privatization. In *Proc. 20th Annual ACM Symposium on Principles of Programming Languages*, January 1993.
15. A. Rogers and K. Pingali. Compiling for locality. In *Proceedings of the 1990 International Conference on Parallel Processing*, pages 142–146, June 1990.
16. C.-W. Tseng. *An Optimizing Fortran D Compiler for MIMD Distributed-Memory Machines*. PhD thesis, Rice University, January 1993. Published as Rice COMP TR93-199.

17. P.-S. Tseng. *A Parallelizing Compiler for Distributed Memory Parallel Computers*. PhD thesis, Carnegie Mellon University, May 1989. Published as CMU-CS-89-148.
18. M. E. Wolf and M. S. Lam. A data locality optimizing algorithm. In *Proceedings of the SIGPLAN '91 Conference on Programming Language Design and Implementation*, pages 30–44, June 1991.
19. M. E. Wolf and M. S. Lam. A loop transformation theory and an algorithm to maximize parallelism. *Transactions on Parallel and Distributed Systems*, 2(4):452–470, October 1991.
20. M. J. Wolfe. *Optimizing Supercompilers for Supercomputers*. MIT Press, Cambridge, MA, 1989.
21. H. P. Zima, H.-J. Bast, and M. Gerndt. SUPERB: A tool for semi-automatic MIMD / SIMD parallelization. *Parallel Computing*, 6(1):1–18, January 1988.